Job Career Assistant **Demo v2**

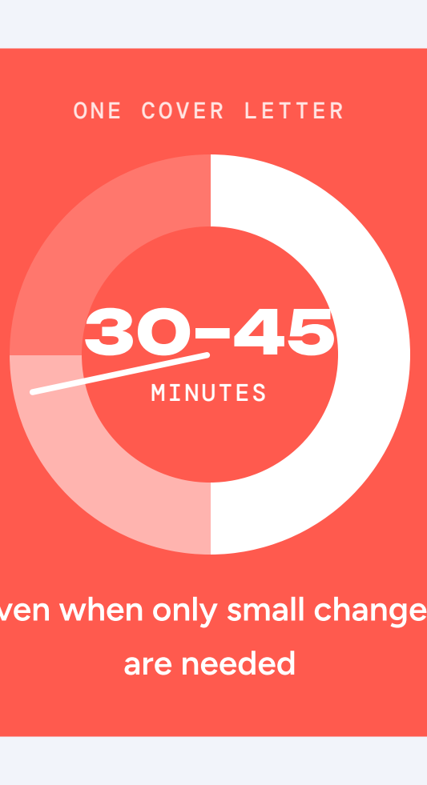
Supervisor demo · v3.0 app

Section 1 · Problem statement and solution

Applying in Germany → one letter at a time is too slow

Recruiters in Germany expect a cover letter written for their needs and requirements. Doing that by hand, for every job, eats the time and energy I need for everything else.

ONE COVER LETTER



even when only small changes are needed

What that does to a day

🟥 4 custom letters ▢ search, interview prep, records

4 letters = 3 hours gone

Example for an 8-hour day at 45 minutes per letter.

PROBLEM
Every letter must be customised
30–45 minutes per letter, even for minor changes. Time and energy go into rewriting.

→ SOLUTION
Letters from my own templates
The app fills only the gaps about the job, in German or English, and checks them so nothing is invented.

PROBLEM
Missed chances and deadlines
While I write one letter, other jobs close and I apply to fewer of them.

→ SOLUTION
Search and apply faster
Ask in plain words, get matching jobs with sources, or paste a real ad from the Jobsuche.

PROBLEM
No energy left for interviews
I am entering the German job market with B2 and C1 German and need to prepare interviews in German and English.

→ SOLUTION
Time back for preparation
The writing is done in minutes, so my energy goes into interviews and language practice.

PROBLEM
A must-do record for the Agentur für Arbeit
Every application has to be recorded and shown to the Agentur für Arbeit.

→ SOLUTION
One click to the table
"Save it" records the application; the app makes one German table, **Bewerbungen.pdf**, for my Berater.

THE SOLUTION

Job Career Assistant

An app that helps me **search jobs** and **write cover letters without fabrication or hallucination**, in German and English, for several Target Roles at once.

DE · Deutsch EN · English

WHAT IT GIVES ME

REPORTS FOR THE AGENTUR FÜR ARBEIT
My must-do record, made for me
When I say "save it", the application is recorded with today's date. One request gives the German table **Bewerbungen.pdf** of all my applications, ready for my Berater.

`Datum` `Arbeitgeber` `Stelle` `Ort` `Vollzeit/Teilzeit` `Präsenz/Online`

TIME REDUCTION
From writing to checking

BEFORE
~~30–45 min~~
writing each letter by hand
→
NOW
one message
"write the cover letter", then I review it

The app fills only the job-specific gaps in my own template and checks them. The table for the Agentur also costs no extra time.

The time saved per letter is not measured yet.

TARGET ROLES

Grant Specialist · AI Engineering · AI Innovation Manager · Quantum Developer · and more

BUILT WITH

LangChain agent, models, memory	**RAG** search over job postings	**Tools** search, letters, Applications	**Streamlit** the chat app

> **Say:** *Applying in Germany means a custom cover letter for every job. One letter takes me 30 to 45 minutes, so I lose deadlines and the energy I need for interviews in German and English, and I also have to record every application for the Agentur für Arbeit. The Job Career Assistant searches jobs and writes my letters in German and English without inventing facts. It turns 30 to 45 minutes of writing into one message and a quick review, and it prepares the report of my applications for the Agentur für Arbeit.*

Section 2 · RAG implementation

How the app finds the right jobs

RAG (retrieval-augmented generation) means the app first searches real job postings, then answers only from what it found.

1 · THE DATA

Why RAG makes sense here

124,000+ job postings in the Kaggle dataset

↓ filtered

1,024 postings for my Target Roles

Far too many postings for one prompt, so the app searches them. I filtered to my roles so I don't spend more API credit on embeddings than I need.

SOURCE
The **Kaggle LinkedIn job postings** dataset: **123,849 postings**, stored in `data/jobs.sqlite`, a SQLite database.

2 · STORED VS. SEARCHABLE

Not everything is searched

STORED, NOT SEARCHABLE
Database
untagged postings and pasted ads (v2)

SEARCHABLE
ChromaDB
the filtered postings, as chunks

Untagged postings and the ads I paste in v2 stay in the database, so the app can still show them, but the search never returns them.

3 · RETRIEVAL

Similarity search = cosine search in ChromaDB

My question → Self-query: filters → Multi-query: rewordings → Cosine search in ChromaDB → RRF merges → Reranker re-scores → Top jobs with sources

```
"remote AI engineer jobs"
↳ "machine learning engineer, remote"
↳ "work-from-home LLM developer"
```

Cosine search finds the chunks whose vectors point in the most similar direction to the question's vector, so it matches by meaning, not exact words. Four steps make it better:

- **Self-query** turns words like "remote" or "in Texas" into hard filters.
- **Multi-query** asks the same question in several ways, so fewer good jobs are missed.
- **RRF** (Reciprocal Rank Fusion) merges the result lists: a job that ranks high in several lists rises to the top.
- A **reranker** reads question and posting together and re-scores the best results.

All in `rag/retriever.py`.

4 · SPLITTER

Cutting postings into chunks

chunk 1 | chunk 2 | chunk 3

CHUNK_SIZE	CHUNK_OVERLAP	UNIT
1,000	**150**	**characters**

LangChain's `RecursiveCharacterTextSplitter` cuts at paragraphs and sentences first. The striped overlap keeps a sentence from being lost at a cut. Done in `rag/indexer.py:22`.

5 · EMBEDDINGS

Text becomes numbers

"AI Engineer, Berlin, hybrid …" → `[0.12, -0.48, 0.07, …]` → **ChromaDB**

MiniLM local · Gemini · Qwen3

An embedding turns text into a vector, a list of numbers, so similar meanings sit close together. One of three models does it, chosen in the sidebar. Defined in `rag/embeddings.py`.

> **Say:** *From more than 124,000 Kaggle postings I filtered 1,024 for my Target Roles, to keep API costs down. They are cut into 1,000-character chunks with 150 overlap, turned into vectors by MiniLM, Gemini or Qwen3, and stored in ChromaDB. Similarity search is a cosine search in ChromaDB, improved with self-query, multi-query, RRF and a reranker, all in rag/retriever.py.*

Section 3 · Tools

9 tools, one agent

Built with the LangChain framework. The agent reads my message and picks the right tool.

LangChain agent
picks a tool for each message

v1 · 5 tools v2 · 4 tools

1 `search_jobs` — v1
Finds matching postings in the Knowledge Base (similarity search + reranker), returns a numbered list with links.
you: remote AI engineer jobs

2 `get_job_description` — v1
Shows the full original posting text, unchanged, in its own box.
you: Show me the job description of #1

3 `create_job_info` — v1
Job Info card: job name, company, work type, role summary, company vision, plus a judge check on each summary.
you: Tell me about #1

4 `write_cover_letter` — v1 · v2: German
Writes the letter from my fixed template, runs the checks, offers downloads.
you: Write a cover letter for #1

5 `job_market_stats` — v1
Exact numbers from the database, shown as a chart (per role, work type, remote, location, salary).
you: How many postings are there per target role?

6 `change_job_facts` — v2
Changes the Target Role, letter language or address of a pasted ad.
you: Write it in English

7 `record_application` — v2
Saves a pasted ad as an Application, dated today (no duplicates).
you: I applied, save it

8 `remove_application` — v2
Deletes a wrong Application; the posting itself stays stored.
you: Remove the application

9 `application_table` — v2
German table of all Applications for the Berater, with a Bewerbungen.pdf download.
you: Give me the Applications PDF

> **Say:** *The app uses the LangChain framework with one agent and 9 tools: five from v1 for searching, explaining jobs, writing letters and statistics, and four from v2 for pasted ads and my Applications for the Agentur für Arbeit.*

Section 4 · Memory

How the app remembers

LangChain has three kinds of memory. Each card shows what LangChain means by it, and what I did in my project.

LANGCHAIN
Message Histories
Store and retrieve conversation history.
IN MY PROJECT
✓ Yes
The `messages` in the agent state hold the chat, and the **last 20 messages** are sent to the model.

LANGCHAIN
Checkpointers
Persist state across sessions.
IN MY PROJECT
✓ Yes
The `SqliteSaver` keeps **every Conversation across restarts**.

LANGCHAIN
Memory Integration
Connect to databases or vector stores for long-term memory.
IN MY PROJECT
Not for conversations
ChromaDB is connected for RAG search over the job postings, not as long-term memory of the chats.

> **Say:** *I use two of LangChain's three kinds of memory for the chat: message history, where the last 20 messages go to the model, and a SqliteSaver checkpointer that keeps every Conversation across restarts. ChromaDB is connected too, but for searching job postings, not as memory of the chats.*

Section 5 · Prompts and security measures

Clear instructions, safe gates

The prompts tell each LLM call exactly what to do. The security measures check every message before the agent sees it.

Prompts

12 prompts, one Markdown file each in `prompts/`, never written inside the code. All can be read on the Evaluation tab.

CHAT: `system.md` `rag_answer.md`
SEARCH: `self_query.md` `multi_query.md`
WRITING: `job_info.md` `cover_letter.md` `job_ad_check.md` `job_facts.md`
CHECKING: `judge.md` `tool_judge.md` `scope_guard.md` `injection_guard.md`

- ✓ **Structured answers:** the LLM must reply in a fixed Pydantic shape, so nothing is parsed from free text.
- ✓ **Data, not orders:** job postings and my messages go in marked blocks, and the prompt says to ignore any instructions inside them.
- ✓ **No facts about me:** the LLM only fills the job-specific gaps; everything about me comes from my own templates.

Security measures

Layers, one after another. If one misses, the next can still catch it. No setting turns any of them off.

1	**Input checks** empty, over 2,000 characters (30,000 for an ad), control characters	REJECT
2	**Injection Guard** (v3) an LLM spots attacks in messages and pasted ads, in any language	BLOCK
3	**Injection patterns** known attack phrases	FLAG + LOG
4	**Scope guard** anything not about job search, e.g. a recipe	REFUSE
5	**Limits and privacy** max 6 tool calls per question; my details never reach the chat model and are removed from the security log	CONTAIN

10/10 attacks blocked

0/10 honest inputs blocked

1 log every event on the Evaluation tab

Security categorisation (if asked "what kind of security is this?")

1. By threat OWASP Top 10 for LLM Applications, 2025

OWASP RISK	WHAT IT MEANS HERE	WHAT THE APP DOES	WHERE
LLM01 Prompt injection, direct	I type text that tries to override the bot's rules	**Injection Guard** blocks it (v3); regex patterns flag and log it	`security.py`, `prompts/injection_guard.md`
LLM01 Prompt injection, indirect	Hidden text in a pasted job ad steers the bot, judge or letter	The ad is checked by the Injection Guard **before** it is saved; it reaches the model only inside a `<job_posting_data>` data block	`agent/chat.py`
LLM02 Sensitive information disclosure	My name, phone, address leaking	My details reach only the Cover Letter step, never the chat model; replaced by `[personal data]` in log excerpts; the profile stays on my laptop	`security.py` (`redacted`), `profile.py`
LLM07 System prompt leakage	"Show me your system prompt"	Blocked by the Injection Guard	`security.py`
LLM06 Excessive agency	The agent doing too much on its own	Max 6 tool calls per question; `job_market_stats` takes fixed choices, so the LLM never writes SQL; Kaggle jobs cannot be saved as Applications	`agent/agent.py`, `tools/stats.py`
LLM09 Misinformation	Invented facts about me or the employer	Facts about me only from my templates; job name and company copied exactly and checked in code; a judge model checks what the LLM writes	`cover_letter.py`, `judge.py`, `job_facts.py`
LLM10 Unbounded consumption	Huge inputs or endless loops costing money	2,000 characters per message, 30,000 per ad; tool-call cap; only the last 20 messages go to the model	`security.py`, `agent/memory.py`
Misuse (not OWASP)	Using the bot for anything else	Scope guard refuses off-topic requests	`security.py`

Also: the API key is in a git-ignored `.env`, and a local pre-commit hook stops personal details and keys from being committed. Typed errors show safe messages, never stack traces or secrets.

2. By how it decides fixed rules vs. LLM checks

TYPE	EXAMPLES	STRENGTH	WEAKNESS
Fixed rules same result every time	Length limits, control characters, regex patterns, tool-call cap, exact field checks, redaction	Free, fast, testable, cannot be talked out of it	Only catches what I thought of: a German or reworded attack slips past the regex
LLM checks a model judges the meaning	Injection Guard, scope guard, judge	Catches new wordings and other languages	One small call each, not 100 % predictable, so it must be measured: the 20-input set

That is why both are used: the fixed rules are a floor that cannot be talked around, and the LLM checks catch what the rules miss.

3. By what it does prevent, detect, limit, fail closed

- **Prevent:** stops the input before harm. Input limits, Injection Guard (blocks), scope guard (refuses), a blocked ad is not saved.
- **Detect:** notices and records. Regex flag, the security log on the Evaluation tab.
- **Limit damage:** keeps harm small if something gets through. Data blocks around ads, tool-call cap, 20-message window, my details never sent to the chat model.
- **Fail closed:** when something breaks, stop instead of skipping. If the Injection Guard's call fails, the turn stops with an error.

How the layers fit together defense in depth

```
my message (+ pasted ad / PDF)
 1. Input limits          fixed rule  → reject     (empty, too long, control characters)
 2. Injection Guard (v3)  LLM         → BLOCK      (ad first, then message; fails closed)
 3. Regex patterns        fixed rule  → flag + log
 4. Scope guard           LLM         → refuse     (off topic)
 5. Agent runs: ad only inside a data block, ≤ 6 tool calls, last 20 messages
 6. Output checks: exact fields + judge model
 7. Security log: every event, personal data removed
```

No setting or dev mode turns any layer off, and a test proves it. If one layer misses, the next can still catch it.

> **Say:** *In one sentence: the app uses layered security, fixed rules plus LLM checks, against the main OWASP LLM risks (prompt injection, data leaks, excessive agency, invented facts and runaway cost). It all runs in* `agent/chat.py` *before the agent, with the rules in* `security.py`.

> **Say:** *Every prompt is its own Markdown file, and the LLM always answers in a fixed structure. Every message passes five security layers before the agent sees it: input checks, the Injection Guard, attack patterns, a scope guard, and limits that protect my data. In the test, the guard blocked all 10 attacks and none of the 10 honest inputs.*

Section 6 · Evaluation

Measured, not guessed

Three evaluations were made: the RAG answers, the agent's tool calling, and the embedding models with and without the reranker.

1 RAGAs evaluation

Faithfulness Does the answer stick to the retrieved chunks?	**0.87–0.99**
Context precision Are the retrieved chunks relevant?	**0.35–0.66**
Context recall Did retrieval find everything needed?	**0.17–0.31**

18 approved questions, 6 setups. Range over all setups; 1 is best.

2 Tool-calling evaluation

Tool selection Did the agent pick the right tool? 10 of 10 cases.	**100 %**
Parameter extraction Did it pass the right arguments? 9 of 10 cases.	**90 %**
Answer quality A second model labels every summary and letter gap "supported" or "fabricated".	**LLM judge**

10 fixed test questions; only the first tool call is scored.

3 Embedding models with and without the reranker

EMBEDDING MODEL	FAITHFULNESS	CONTEXT PRECISION	CONTEXT RECALL	RERANKER EFFECT
MiniLM (local)	0.87 → 0.99 +0.12	0.45 → 0.66 +0.21	0.21 → 0.31 +0.10	Clear gain in all three
Gemini Embedding 2	0.93 → 0.93 ±0	0.35 → 0.59 +0.24	0.17 → 0.29 +0.12	Biggest precision gain
Qwen3-Embedding-8B	0.95 → 0.94 -0.01	0.49 → 0.50 +0.01	0.17 → 0.20 +0.03	No clear difference

- Read each cell as **without** → **with** the reranker, on the same 18 questions and the same queries.
- The reranker helps most where the embedding's own ranking is weakest: Gemini +0.24 and MiniLM +0.21 in precision.
- Qwen3 already ranks well on its own, so the reranker adds little; its changes are smaller than the run-to-run noise of 0.1–0.2.
- The reranker runs locally, so it adds no cost per question, only about 44 seconds of loading at start.

> **Say:** *I made three evaluations. RAGAs shows the answers stay faithful to the postings, 0.87 to 0.99, while recall is low. The agent picked the right tool in 10 of 10 cases and the right arguments in 9 of 10, and a judge model checks every written summary. The reranker clearly raised precision for MiniLM and Gemini; for Qwen3 it made no clear difference.*